Universidade de Aveiro
Exame Teórico-Prático 2 (ATP2) de Base de Dados — **6 de Junho 2025**
Duração: 60 minutos

[redacted]

Relativamente às perguntas 1 a 30, assinale na tabela ao lado, um X na coluna "V" para as declarações que estão corretas e na "F" para as que estão incorretas.

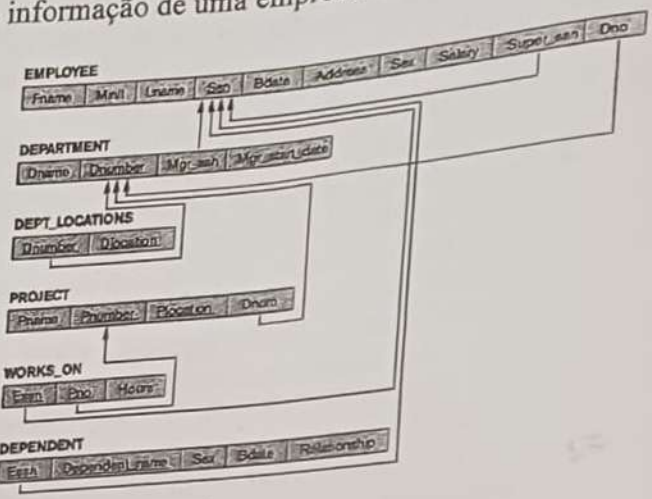
Cada uma destas perguntas vale 0,5 valores e cada resposta errada desconta 0,25 valores.
[15]

1. Em SQLServer, os índices *non-clustered* podem existir em qualquer tabela, independentemente de esta ser *clustered* ou *heap*.
2. Em SQLServer, o *fill factor* permite reservar espaço livre em páginas de índices para reduzir a ocorrência de *page splits*.
3. A criação de um índice multi-atributo permite acelerar pesquisas mesmo que não se use o primeiro atributo na cláusula WHERE.
4. A existência de muitos índices numa tabela nunca afeta negativamente o desempenho de operações de update e delete.
5. Um *heap* table no SQLServer mantém sempre os dados ordenados fisicamente segundo a ordem da chave primária.
6. Quando uma *batch* termina, todas as variáveis locais declaradas nela são automaticamente destruídas.
7. Em SQLServer, um erro de runtime numa batch anula automaticamente todas as instruções já executadas nessa batch.
8. As variáveis do tipo tabela (@Table) não podem ter chaves estrangeiras ou triggers associados.
9. Cursores permitem percorrer tuplos de uma tabela um a um, o que em geral é mais eficiente que soluções set-based.
10. Triggers INSTEAD OF substituem completamente a execução da instrução DML original.
11. As funções definidas pelo utilizador (UDF) podem conter instruções de controlo de fluxo como TRY...CATCH.
12. As tabelas lógicas INSERTED e DELETED podem ser diretamente consultadas dentro de stored procedures chamadas a partir de triggers.
13. O comando RAISERROR pode ser usado dentro de triggers e stored procedures para forçar mensagens de erro, mas não faz o rollback da transação.
14. Em SQLServer, @@ERROR e @@ROWCOUNT reportam respetivamente o código de erro e o número de linhas afetadas.
15. O estado *Partially Committed* ocorre imediatamente antes do *commit* ser garantido fisicamente no disco.
16. A propriedade *Durability* do modelo ACID garante que nenhuma falha posterior pode anular as alterações de uma transação com *commit* efetuado.
17. Num escalonamento recuperável, uma transação T2 pode fazer *commit* mesmo que tenha lido dados atualizados por T1, ainda não terminado.
18. O método otimista de controlo de concorrência nunca provoca *aborts* de transações, apenas atrasos de execução.
19. Em SQLServer, o *transaction log* pode ser eliminado periodicamente após a execução de *commits*.
20. A utilização de *Savepoints* permite efetuar *rollbacks* parciais dentro de uma transação.
21. O *checkpoint* armazena no *transaction log* todas as queries executadas após o último *commit*, servindo como histórico completo de recuperação.
22. Em SQLServer, um *user* pode pertencer simultaneamente a múltiplas *database roles*.

	V	F
1		
2		
3		
4		
5		
6		
7		
8		
9		
10		
11		
12		
13		
14		
15		
16		
17		
18		
19		
20		
21		
22		
23		
24		
25		
26		
27		
28		
29		
30		

23. O comando DENY tem sempre prioridade sobre GRANT na resolução de permissões.
24. A recuperação de falha de disco utiliza os backups e aplica as *before-images* do *transaction log*.
25. SQL Injection não permite executar instruções DDL como, por exemplo, DROP TABLE.
26. Uma relação com chave simples (não composta) não pode violar a 2FN.
27. Se uma relação está na 3FN, não pode existir qualquer dependência funcional entre atributos não-chave.
28. Na decomposição em BCNF nunca há perda de dependências funcionais.
29. No processo de normalização de uma base de dados relacional, a redundância está associada a dependências funcionais não desejadas.
30. Em SQL, as *Views* podem ser parametrizadas.

31. *[5]* Considere a script abaixo para o sistema de informação de uma empresa usado nas aulas. Esta não contém erros de sintaxe.




```
go
CREATE TYPE ProjectListType AS TABLE (
    Pno INT
);


go
CREATE PROCEDURE TransferEmployeeDepartment
    @Ssn CHAR(9),
    @NewDno INT,
    @ProjectList ProjectListType READONLY
AS
BEGIN
    BEGIN TRY
        BEGIN TRANSACTION

        IF NOT EXISTS (SELECT 1 FROM EMPLOYEE WHERE Ssn = @Ssn)
        BEGIN
            RAISERROR ('Error .....', 16, 1)                    -- linha X
            ROLLBACK TRAN
            RETURN
        END

        IF NOT EXISTS (SELECT 1 FROM DEPARTMENT WHERE Dnumber = @NewDno)
        BEGIN
            RAISERROR ('Error .....', 16, 1)
            ROLLBACK TRAN
            RETURN
        END

        IF EXISTS (
            SELECT 1 FROM DEPARTMENT
            WHERE Mgr_ssn = @Ssn AND Dnumber <> @NewDno

        BEGIN
            RAISERROR ('Error .....', 16, 1)
            ROLLBACK TRAN
            RETURN
        END

        UPDATE EMPLOYEE SET Dno = @NewDno WHERE Ssn = @Ssn

        DELETE FROM WORKS_ON WHERE Essn = @Ssn

        DECLARE @Pno INT

        DECLARE proj_cursor CURSOR FOR
            SELECT Pno FROM @ProjectList

        OPEN proj_cursor
        FETCH NEXT FROM proj_cursor INTO @Pno
```

```
    WHILE @@FETCH_STATUS = 0
    BEGIN
      IF NOT EXISTS (SELECT 1 FROM Project WHERE Pnumber = @Pno)
      BEGIN
        RAISERROR ('Error .....', 16, 1);
        ROLLBACK TRAN
        RETURN
      END

      INSERT INTO WORKS_ON (Essn, Pno, Hours) VALUES (@Ssn, @Pno, 0)
      FETCH NEXT FROM proj_cursor INTO @Pno
    END

    CLOSE proj_cursor
    DEALLOCATE proj_cursor

    COMMIT TRAN
  END TRY
  BEGIN CATCH
    RAISERROR ('Error .....', 16, 1);
    ROLLBACK TRAN
  END CATCH
END
go

---------------------------------------------------------------------------------------------- test

DECLARE @Ssn VARCHAR(9)
@Ssn = '123456789'

DECLARE @Projects ProjectListType;
INSERT INTO @Projects (Pno) VALUES (10), (20), (30);

EXEC TransferEmployeeDepartment @Ssn = '123456789', @NewDno = 5, @ProjectList = @Projects;
```

Das seguintes afirmações, assinale as 5 são verdadeiras. Cada errada anula uma coreta.

Esta SP garante a integridade dos dados validando a existência de funcionários e departamentos.
Esta SP permite que funcionários trabalhem em vários departamentos.
Esta SP garante que um funcionário não possa gerenciar vários departamentos.
Esta SP impede a exclusão de atribuições de projetos antigas do funcionário.
Esta SP usa um cursor para iterar sobre uma lista de projetos passada como um parâmetro com valor de tabela.
Esta SP ignora projetos inexistentes e continua com os demais.
Esta SP insere 0 horas de trabalho ao reatribuir um funcionário a novos projetos.
Esta SP permite a inserção de números de projetos inválidos.
Esta SP confirma as alterações mesmo se um erro for gerado durante a inserção do projeto.
Esta SP suporta a reatribuição de funcionários a vários departamentos em uma única execução.
Esta SP gera um erro se um projeto na lista não existir na tabela PROJETO.
Este SP atribui funcionários como gerentes do novo departamento.
Este SP verifica se o funcionário já trabalha nos novos projetos antes da inserção.
Este SP registra mensagens de erro detalhadas do sistema usando ERROR_MESSAGE().
Este SP permite a inserção de horas negativas em WORKS_ON.

* NOTA: Opções geradas pelo Chat, podem nem haver as 5 corretas que diz no enunciado mas a pergunta tinha esta estrutura.

(A página 4/4 era toda de rascunho)